ASE CHAPTER 1

T1 · Multiple Choice Questions

Official MCQs 1.1–1.20 with sourced answers · plus an authored set in the same pattern

Rapid self-test — recall prompts (no answers shown)

> Cover the cards above, then answer these from memory. If any item fails, return to its source page.

Define software in its three-component form.
Name the two families of manuals and give two members of each.
List the four software characteristics.
Distinguish product from process in one sentence each.
List the three process metrics and three product metrics.
State the unit of effort, and justify why it is neither "persons" nor "months".
Name the four factors of management dependency **in the required order**.
State the two definitions of software engineering quoted in the text.
Give the four reasons software process improvement fails.
State the percentage of development projects attributed to customized products.

Retrieval set — Part A · Official MCQs (1.1 – 1.20)

1.1 · Software is

(a) Superset of programs · **(b)** Subset of programs · **(c)** Set of programs · **(d)** None of the above

Superset of programs (a) — Answer

Software = Program + Documentation + Operating Procedures. Programs are one component inside software — software is the **superset**, the program the **subset**. Option (b) reads the same relationship backwards.

Aggarwal, §1.2.2, pp.4–5 (Fig. 1.1).

1.2 · Which is NOT the part of operating procedure manuals?

(a) User manuals · **(b)** Operational manuals · **(c)** Documentation manuals · **(d)** Installation manuals

Documentation manuals (c) — Answer

Two separate families.

Operating procedure manuals: user manuals, operational manuals, system overview, beginner's guide tutorial, reference guide, installation guide, system administration guide.

Documentation manuals: analysis/specification, design, implementation, testing, formal specification, context diagram, data flow diagrams, flow charts, entity-relationship diagram, source code listings, cross-reference listing, test data, test results.

Aggarwal, §1.2.2, pp.5–6 (Fig. 1.2, Fig. 1.3).

1.3 · Which is NOT a software characteristic?

(a) Software does not wear out · **(b)** Software is flexible · **(c)** Software is not manufactured · **(d)** Software is always correct

Software is always correct (d) — Answer 📝

Characteristics: software **does not wear out**, software **is not manufactured**, software **is flexible**, and its components are **reusable**. "Always correct" is not a property — it is the exact opposite of what the myths section later warns about.

Aggarwal, §1.2.4, pp.8–10.

ليش (d) هي الفخ ⚡

باقي الخيارات كلها خصائص حقيقية منصوصة، فالسؤال يكافئ اللي **حفظ القائمة**. و«always correct» تسمع مثل مدح — لهذا تعدي على قراءة سريعة. حالات الفشل المذكورة مثل Patriot وY2K توضح عملياً أن البرمجيات مو دائماً صحيحة.

Aggarwal, §1.2.4, pp.8–10; §1.1.1, pp.2–3.

1.4 · Product is

(a) Deliverables · **(b)** User expectations · **(c)** Organization's effort in development · **(d)** None of the above

Deliverables (a) — Answer 📝

"Product: what is delivered to the customer... Basically, it is nothing but a set of deliverables only." Option (c) — the organization's effort in development — is the definition of **process**, not product.

Aggarwal, §1.5.2, p.13.

1.5 · To produce a good quality product, process should be

(a) Complex · **(b)** Efficient · **(c)** Rigorous · **(d)** None of the above

Efficient (b) — Answer 📝

"An efficient process is required to produce good quality products." "If the process is weak, the end product will undoubtedly suffer, but an obsessive over-reliance on process is also dangerous."

Aggarwal, §1.5.2, p.13.

1.6 · Which is NOT a product metric?

(a) Size · **(b)** Reliability · **(c)** Productivity · **(d)** Functionality

Productivity (c) — Answer 📝

"Examples of process metrics include productivity, quality, failure rate, efficiency etc. Examples of product metrics are size, reliability, complexity, functionality etc."

Productivity is a process metric, so it is the one that does not belong. Note that **functionality** is a product metric — which is exactly why it cannot also be the answer here.

Aggarwal, §1.5.4, p.13.

1.7 · Which is NOT a process metric?

(a) Productivity · **(b)** Functionality · **(c)** Quality · **(d)** Efficiency

Functionality (b) — Answer

"Examples of process metrics include productivity, quality, failure rate, efficiency etc." Question 1.6 read in reverse.

Aggarwal, §1.5.4, p.13.

قاعدة واحدة تحل 1.6 و1.7 و1.19 من التمارين

اسأل: **مَن المُقاس؟** إذا القياس يوصف **نشاط البناء** ⇒ مقياس عملية. وإذا يوصف **الشي المبني** ⇒ مقياس منتج. والـproductivity دائماً عملية لأنه **معدّل** (ناتج ÷ جهد).

Aggarwal, §1.5.4–1.5.5, pp.13–14.

1.8 · Effort is measured in terms of:

(a) Person-months · **(b)** Rupees · **(c)** Persons · **(d)** Months

Person-months (a) — Answer

"Most appropriate unit of effort is Person Months (PMs), meaning thereby number of persons involved for specified months." Option (c) counts people, option (d) counts time — effort is the **product** of the two, which is why neither alone works.

Aggarwal, §1.5.5, p.14.

1.9 · UML stands for

(a) Uniform modeling language · **(b)** Unified modeling language · **(c)** Unit modeling language · **(d)** Universal modeling language

Unified modeling language (b) — Answer

"Another definition from unified modeling language (UML) [OMG2K] is: 'A component represents a modular, deployable, and replaceable part of a system that encapsulates implementation and exposes a set of interfaces.'"

Aggarwal, §1.5.6, p.14.

1.10 · An independently deliverable piece of functionality providing access to its services through interface is called

(a) Software measurement · **(b)** Software composition · **(c)** Software measure · **(d)** Software component

Software component (d) — Answer

"An independently deliverable piece of functionality providing access to its services through interfaces." "A component represents a modular, deployable, and replaceable part of a system that encapsulates implementation and exposes a set of interfaces."

Aggarwal, §1.5.6, p.14.

⚠ **هذا السؤال مطبوع `1.1` بسلايد 58 — رقم مكرر**

فهرسته هنا **1.10** حتى يمشي التسلسل 1.1 ← 1.20 بلا فراغ.

Aggarwal, §1.5.6, p.14.

1.11 · Infrastructure software are covered under

(a) Generic products · **(b)** Customized products · **(c)** Generic and Customized products · **(d)** None of the above

Generic products (a) — Answer

"Generic products are developed for anonymous customers... Infrastructure software like operating system, compilers, analyzers, word processors, CASE tools etc. are covered in this category."

Aggarwal, §1.5.7, p.14.

1.12 · Management of software development is dependent on

(a) people · **(b)** product · **(c)** process · **(d)** all of the above

all of the above (d) — Answer

"The management of software development is heavily dependent on four factors: People, Product, Process, and Project." Options (a), (b) and (c) are each one of them.

Aggarwal, §1.6, p.15.

1.13 · During software development, which factor is most crucial?

(a) People · **(b)** Product · **(c)** Process · **(d)** Project

People (a) — Answer

"The management of software development is heavily dependent on four factors: People, Product, Process, and Project." Order of dependency (Fig. 1.8): **People (1)** → **Product (2)** → **Process (3)** → **Project (4)**. People comes first, so it is the most crucial factor.

"The process priority is after people and product, however, it plays very critical role for the success of the project."

Aggarwal, §1.6, p.15; §1.6.3, p.16; Fig. 1.8, p.15.

1.14 · Program is

(a) subset of software · **(b)** super set of software · **(c)** software · **(d)** none of the above

subset of software (a) — Answer

Software = Program + Documentation + Operating Procedures.

Aggarwal, §1.2.2, pp.4–5 (Fig. 1.1).

1.15 · Milestones are used to

(a) know the cost of the project · **(b)** know the status of the project · **(c)** know user expectations · **(d)** none of the above

know the status of the project (b) — Answer

"The milestones are the events that are used to ascertain the status of the project... The milestones are essential for project planning and management." Cost belongs to estimation, and user expectations belong to requirements — neither is what a milestone reports.

Aggarwal, §1.5.1, p.13.

1.16 · The term module used during design phase refers to

(a) Function · **(b)** Procedure · **(c)** Sub program · **(d)** All of the above

All of the above (d) — Answer

No single definition is fixed; a module may be **"a FORTRAN subroutine"**, **"an Ada package"**, **"procedures and functions of PASCAL and C"**, **"C++/Java classes"**, **"Java packages"**, or even **"a work assignment for an individual developer"** — **"all these definitions are correct."**

Aggarwal, §1.5.6, p.14.

1.17 · Software consists of

(a) Set of instructions + operating system · **(b)** Programs + documentation + operating procedures · **(c)** Programs + hardware manuals · **(d)** Set of programs

operating procedures + documentation + Programs (b) — Answer

Software = Program + Documentation + Operating Procedures (Fig. 1.1, p.4). Options (a), (c) and (d) each drop two of the three components.

Aggarwal, §1.2.2, pp.4–5 (Fig. 1.1).

1.18 · Software engineering approach is used to achieve:

(a) Better performance of hardware · **(b)** Error free software · **(c)** Reusable software · **(d)** Quality software product

Quality software product (d) — Answer

Schach: **"a discipline whose aim is the production of** quality software**, software that is delivered on time, within budget, and that satisfies its requirements." "The art of programming only is no longer sufficient to construct large programs."**

Aggarwal, §1.2 and §1.2.1, p.4.

1.19 · Concept of software engineering are applicable to

(a) Fortran language only · **(b)** Pascal language only · **(c)** 'C' language only · **(d)** All of the above

All of the above (d) — Answer

The two definitions rest on **engineering principles** and on the **process** of producing quality software — they name no programming language, so the concepts are not tied to any one language.

Aggarwal, §1.2.1, p.4; Answers, p.498.

سبب اختيار (d)

التعريفان ما يذكران لغةً معينة؛ والمفتاح الرسمي يؤكد الخيار (d).

1.20 · CASE Tool is

(a) Computer Aided Software Engineering · **(b)** Component Aided Software Engineering · **(c)** Constructive Aided Software Engineering · **(d)** Computer Analysis Software Engineering

Computer Aided Software Engineering (a) — Answer

"Many people (especially CASE tool vendors) believe that CASE (Computer Aided Software Engineering) tools represent the so-called silver bullet..." "Operating systems, compilers, analysers, word processors, CASE tools etc."

Aggarwal, §1.1.2, p.3; §1.5.7, p.14.

Part B · Authored MCQs (same pattern)

B.1 · The definition of "software" differs from the everyday use of the word mainly because it includes:

(a) the hardware it runs on · **(b)** the documentation and operating procedures · **(c)** the development team · **(d)** the licence agreement

the documentation and operating procedures (b) — Answer

Software = Program + Documentation + Operating Procedures. The everyday sense of "software" is programs; the definition is a three-part construct, which is why a program alone is not a software product.

Aggarwal, §1.2.2, pp.4–5 (Fig. 1.1).

B.2 · Which of the following belongs to the operating procedure manuals rather than to the documentation manuals?

(a) Design specification · **(b)** Data-flow diagram · **(c)** Installation guide · **(d)** Test results

Installation guide (c) — Answer

The two families are kept strictly apart. Design specifications, data-flow diagrams and test results are all **documentation**; the installation guide is listed among the **operating procedure** manuals.

Aggarwal, §1.2.2, pp.5–6 (Fig. 1.2, Fig. 1.3).

B.3 · A software house builds a payroll system for one specific ministry. Under the classification this product is:

(a) generic · **(b)** bespoke (customized) · **(c)** infrastructure · **(d)** open source

(customized) bespoke (b) — Answer

"The customized products are developed for particular customers." About 80% of development projects fall in this category.

Aggarwal, §1.5.7, p.14.

B.4 · Software process improvement commonly fails for all of the following reasons EXCEPT:

(a) not enough time · **(b)** lack of knowledge · **(c)** wrong motivations · **(d)** the absence of a CASE tool

the absence of a CASE tool (d) — Answer

Four reasons software process improvement is difficult: **"not enough time"**, **"lack of knowledge"**, **"wrong motivations"** and **"insufficient commitment"**. Tooling is not among them, so (d) is the exception.

Keep this separate from the different question of why CASE tools cannot end the crisis — that argument belongs to §1.1.2 and rests on the no-silver-bullet claim, not on this list. The two sections answer two different questions.

Aggarwal, §1.2.3, pp.6–7.

B.5 · Which statement is one of the developer-perspective software myths?

(a) The tested code is the only deliverable of a project · **(b)** A prototype reduces requirements risk · **(c)** Maintenance costs dominate the life-cycle cost · **(d)** Requirements change during development

The tested code is the only deliverable of a project (a) — Answer

Developer-perspective myth: **"Tested code is only one of the deliverable!"** Options (b), (c) and (d) are supported positions, not myths.

Aggarwal companion slides, slide 45 (Software Myths — Developer Perspectives).

B.6 · The aim of software development has shifted from producing working programs to producing:

(a) smaller programs · **(b)** good quality maintainable programs · **(c)** programs with more features · **(d)** error-free programs

good quality maintainable programs (b) — Answer

"The aim has been shifted from developing working programs to good quality, maintainable programs." The change was driven by the **"cost of maintaining software"**.

Aggarwal, §1.4, p.12; §1.2.1, p.4.

B.7 · "The strength and stability of a bridge can be calculated with reasonable precision" is listed as a difference between building a bridge and writing a program. The corresponding statement for a program is:

(a) the correctness of a program can be calculated with existing methods · **(b)** it is not possible to calculate the correctness of a program with existing methods · **(c)** every program is a repeat of a previous program · **(d)** the requirements for a program do not change during construction

program with existing it is not possible to calculate the correctness of a (b) — Answer methods

The bridge-versus-program comparison pairs every engineering strength with a software weakness: the bridge problem is **well understood**, the program problem is **only partly understood**; bridge requirements **barely change**, program requirements **change in all phases**; bridges are **calculable**, programs are **not**.

Aggarwal, §1.2.4, Table 1.1, p.8.

B.8 · If productivity is defined as output per unit of effort, and effort is measured in person-months, then productivity is measured in:

(a) lines of code · **(b)** person-months · **(c)** lines of code per person-month · **(d)** person-months per line of code

lines of code per person-month (c) — Answer

"Productivity may be measured as LOC/PM (lines of code produced/person month)." Option (d) inverts the ratio — a classic trap, because it is the **effort per unit of output** (the inverse rate), not productivity.

Aggarwal, §1.5.5, p.14.

ASE CHAPTER 1

T2 · Descriptive & Short-Answer (Interpretation) Questions

Exercise 1.1–1.25 answered directly from the source — answers in English, notes in Arabic

Chapter 1 — Introduction

طريقة قراءة الإجابات

الكلام العريض مبني على المصدر، وقد يُبسّط بألفاظ أسهل. الكلام العادي صياغة أو استنتاج مختصر. الاقتباس الحرفي فقط يوضع بين علامتي تنصيص.

Rapid self-test — recall prompts (no answers shown)

State the definition of software, with its three components.
Give the five contributing factors of the software crisis.
State what a silver bullet is, and why CASE tools are not one.
Name the three difficulties that make software technology's progress "not that rosy".
Give the four reasons why software process improvement is difficult.
List the four software characteristics, and name the one blamed for the crisis.
Distinguish measure, measurement and metric in one line each.
Name the eight application areas of software.
State the unit of effort, and give the four factors of management dependency **in the numbered order**.
State the percentage of development projects attributed to customised products — and say precisely what that percentage is a share of.

Retrieval set — Official Exercises (1.1 – 1.25)

1.1 · Why is the primary goal of software development now shifting from producing good quality software to good quality maintainable software?

Answer

Because the cost of maintaining software increased, the objective is now to produce software that is maintainable, delivered on time, within budget, and that satisfies its requirements.

Aggarwal, §1.2.1, p.4.

1.2 · List the reasons for the "software crisis"? Why are CASE tools not normally able to control it?

Answer

Reasons. Change in the ratio of hardware to software costs · increasing importance of maintenance · advances in software techniques · increased demand for software · demand for larger and more complex software systems.

Why CASE tools cannot control it. A tool is not a silver bullet: **"no single development, either in technology or in management technique, that by itself promises even one order-of-magnitude improvement in productivity, in reliability, in simplicity." "Many companies have used these tools and spent large sums of money, but results were highly unsatisfactory." "The hard part of building software is the specification, design and testing of this conceptual construct, not the labour of representing it."**

Aggarwal, §1.4, p.12; §1.1.2, p.3.

1.3 · "The software crisis is aggravated by the progress in hardware technology?" Explain with examples.

Answer

The chapter contrasts rapid hardware gains with slower software progress: **"No other technology, since civilization began, has seen six orders of magnitude in performance-price gain in 30 years. The progress in software technology is not that rosy due to certain difficulties with this technology. Some of the difficulties are complexity, changeability and invisibility."** Y2K and Ariane-5 illustrate software failures; they do not by themselves prove that hardware progress caused the crisis.

Two examples from the failure list: **Y2K** — the four-digit date **"was shortened to 2-digit format, like 64"**; and **Ariane-5** — the guidance computer converting the sideways velocity **"from a 64-bit format to a 16-bit format"** produced an overflow after 36.7 seconds.

Aggarwal, §1.1.2, p.3; §1.4, p.12; §1.1.1, pp.2–3.

⚠ فخ هذا السؤال

لا تخلط بين تزامن تقدّم العتاد ومشكلات البرمجيات وبين إثبات علاقة سببية مباشرة. تغيّر نسبة كلفة العتاد إلى البرمجيات عامل مستقل يورده الكتاب ضمن أسباب الأزمة.

1.4 · What is software crisis? Was Y2K a software crisis?

Answer

"Software crisis has become a fixture of everyday life" — late delivery, cost overrun, residual faults, failures causing **"death of many human beings"**. **"This unpredictability is the basis of what has been referred to for the past 30 years as the 'Software Crisis'."**

Y2K — "the most crucial problem of last century": **"It was simply the ignorance about the adequacy or otherwise of using only last two digits of the year... Millions of rupees have been spent to handle this practically non-existent problem."**

Aggarwal, §1.1, pp.1–2; §1.2.4, p.10.

⚠ لا تكتب إن Y2K دمّرت أنظمة

الكتاب يسمي المشكلة نفسها **"practically non-existent"**، ويذكر أن الملايين صُرفت لمعالجتها. الضرر المذكور هنا هو كلفة المعالجة؛ لا تضف انهيار أنظمة ما لم يذكره المصدر.

1.5 · What is the significance of software crisis in reference to software engineering discipline?

The crisis is the reason the discipline exists: **"These myths, poor quality of software, increasing cost and delay in the delivery of the software have been the driving forces behind the emergence of software engineering as a discipline."**

Aggarwal, §1.4, p.12.

1.6 · How are software myths affecting software process? Explain with the help of examples.

Answer 📝

"There are number of myths associated with software development community. Some of them really affect the way, in which software development should take place." Nine myths:

1. Software is easy to change. "Every change requires that the complete system be re-verified."

2. Computers provide greater reliability than the devices they replace. "Now, we have software error as well."

3. Testing or 'proving' software correct can remove all the errors. "Testing can only show the presence of errors. It cannot show the absence of errors."

4. Reusing software increases safety. "A false sense of security."

5. Software can work right the first time. "No aeronautical engineer would build a jet fighter without a prototype."

6. Software can be designed thoroughly enough to avoid most integration problems. "There is an old saying among software designers: 'Too bad, there is no compiler for specifications'."

7. Software with more features is better software. "The best, most enduring programs are those which do one thing well."

8. Addition of more software engineers will make up the delay. It **"may further delay the project."**

9. Aim is to develop working programs. "The aim has been shifted from developing working programs to good quality, maintainable programs."

Aggarwal, §1.4, pp.11–12.

1.7 · State the difference between program and software. Why have documents and documentation become very important?

Software **"is more than programs"**: it **"consists of programs, documentation of any facet of the program and the procedures used to setup and operate the software system."** A program **"is a combination of source code and object code"**, and **"any program is a subset of software and it becomes software only if documentation and operating procedure manuals are prepared."**

Why documentation matters. Documentation explains system design and operation; operating procedures guide setup and use. These materials help users and maintainers understand the system beyond its source and object code.

Aggarwal, §1.2.2, pp.4–5; Fig. 1.2 p.5, Fig. 1.3 p.6.

توضيح

ذكر الصيانة هنا تطبيق عملي لأهمية التوثيق؛ المصدر يعرّف مكونات البرمجيات ويصف دور الإجراءات في إعداد النظام وتشغيله.

1.8 · What is software engineering? Is it an art, craft or a science? Discuss.

Answer

Definitions. Fritz Bauer, at the 1968 conference [FRIT68]: **"The establishment and use of sound engineering principles in order to obtain economically developed software that is reliable and works efficiently on real machines."** Stephen Schach [SCHA90]: **"A discipline whose aim is the production of quality software, software that is delivered on time, within budget, and that satisfies its requirements."**

Art, craft or science. Software engineering combines systematic engineering principles, scientific knowledge, and design judgement. Large programs require more than programming skill alone: **"The art of programming only is no longer sufficient to construct large programs"**, and software engineering is an important discipline of study, practice and research.

Aggarwal, §1.2.1, p.4; §1.6.4, p.16.

فخ مباشر

ميّز بين التعريفات الواردة في المصدر وبين موقفك المعلّل من كونها فناً أو حرفةً أو علماً.

1.9 · What is the aim of software engineering? What does the discipline of software engineering discuss?

Answer

The aim: "Software engineering has the objective of solving these problems by producing good quality, maintainable software, on time, within budget. To achieve this objective, we have to focus in a disciplined manner on both the quality of the product and on the process used to develop the product."

What the discipline discusses: the evolving role of software and its failures · what software engineering is · program versus software · software process · software characteristics · the changing nature of software · software myths · terminology · the role of management.

Aggarwal, §1.2, p.4; §1.1–1.6, pp.1–16.

1.10 · Define the term "Software Engineering". Explain the major differences between software engineering and other traditional engineering disciplines.

Answer

Definitions. Two, both accepted. Fritz Bauer, at the 1968 conference [FRIT68]: **"The establishment and use of sound engineering principles in order to obtain economically developed software that is reliable and works efficiently on real machines."** Stephen Schach [SCHA90]: **"A discipline whose aim is the production of quality software, software that is delivered on time, within budget, and that satisfies its requirements."**

Differences (Table 1.1 — bridge against program): **the problem is well understood / only some parts of the problem are understood, others are not · there are many existing bridges / every program is different and designed for special applications · the requirement for a bridge typically do not change much during construction / requirements typically change during all phases of development · the strength and stability of a bridge can be calculated with reasonable precision / not possible to calculate correctness of a program with existing methods · when a bridge collapses, there is a detailed investigation and report / when a program fails, the reasons are often unavailable or even deliberately concealed · engineers have been constructing bridges for thousands of years / developers have been writing programs for 50 years or so · materials and techniques change slowly / hardware and software changes rapidly**.

Aggarwal, §1.2.1, p.4; Table 1.1, p.8.

1.11 · What is software process? Why is it difficult to improve it?

Answer

"The software process is the way in which we produce software."

Four reasons it is difficult to improve [HUMP89, WIEG99]: **not enough time** — unrealistic schedules leave no room to study what is wrong · **lack of knowledge** — developers do not read the literature on best-known practice · **wrong motivations** — initiatives launched to satisfy a contractor or a manager's CMM ambition · **insufficient commitment** — assessment without follow-through, no plan and no roadmap.

Add Fig. 1.4: improvement **"will not have an impact on current productivity"** — the curve dips before it rises.

Aggarwal, §1.2.3, pp.6–7; Fig. 1.4, p.7.

1.12 · Describe the characteristics of software contrasting it with the characteristics of hardware.

Answer

1. Software does not wear out. Hardware has three phases — burn-in, useful life, wear out (Fig. 1.5). **"We do not have this phase for the software as it does not wear out"**; **"software becomes reliable overtime instead of wearing out"** (one word: overtime). Software instead **"becomes obsolete, if the environment for which it was developed, changes."**

2. Software is not manufactured. "Making 1000 copies is not an issue and it does not involve any cost... We do not have assembly line in software development."

3. Reusability of components. Hardware is assembled from purchased parts; in software **"every project is a new project. We start from the scratch and design every unit"** — hence component based software engineering.

4. Software is flexible. "A program can be developed to do almost anything", which makes development **"difficult to plan, monitor and control"**; **"this unpredictability is the basis of what has been referred to for the past 30 years as the 'Software Crisis'."**

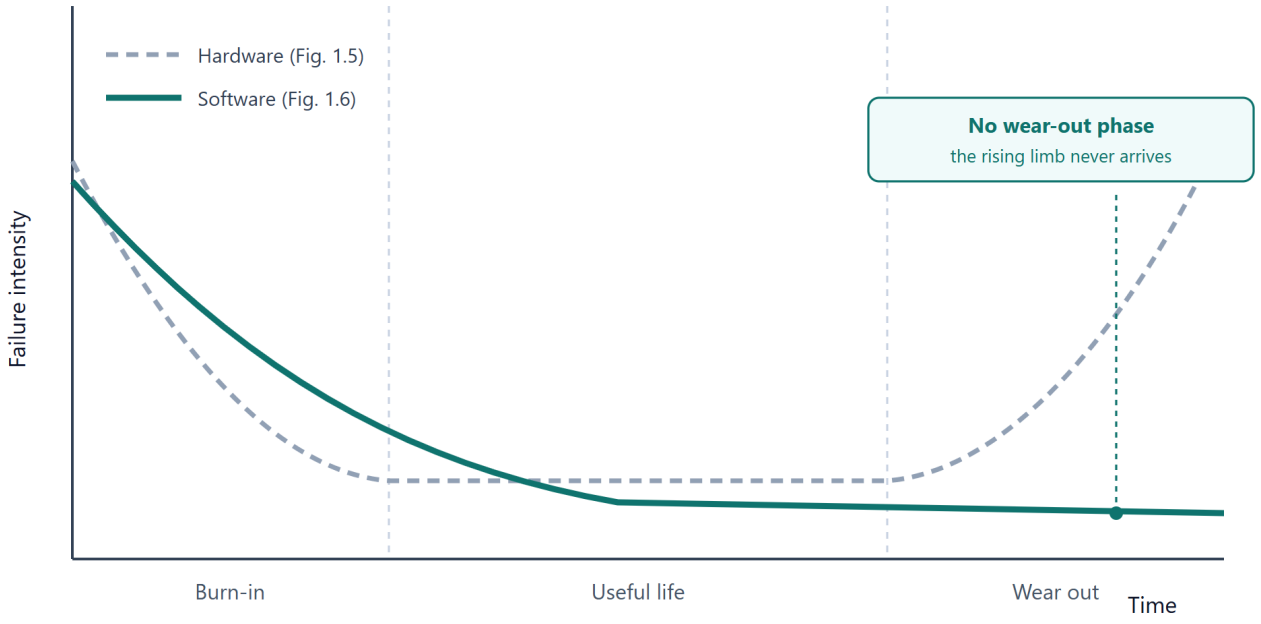
Aggarwal, §1.2.4, pp.8–10; Table 1.1, p.8.

1.13 · Write down the major characteristics of a software. Illustrate with a diagram that the software does not wear out.

Answer

The four characteristics (§1.2.4): software **does not wear out** · software **is not manufactured** · **reusability of components** · software **is flexible**.

The diagram — Fig. 1.6, drawn against the hardware curve in Fig. 1.5. Axes: **failure intensity** against **time**. Hardware has three phases — high intensity in **burn-in**, a flat low plateau through **useful life**, then a **rising** limb at **wear out**. Software shows only the **descending** curve, flattening as defects are found and fixed: the rising limb never arrives, because there is no physical wear-out mechanism.



Failure intensity against time — the hardware bath-tub curve (Fig. 1.5) against the software curve (Fig. 1.6)

Aggarwal, §1.2.4, pp.8–9; Fig. 1.5 p.8, Fig. 1.6 p.9.

1.14 · What are the components of a software? Discuss how a software differs from a program.

Answer

Components (Fig. 1.1): **Software = Program + Documentation + Operating Procedures.**

Documentation manuals (Fig. 1.2) cover analysis/specification (formal specification, context diagram, data flow diagrams), design (flow charts, entity-relationship diagrams), implementation (source code listings, cross-reference listing) and testing (test data, test results). Operating procedure manuals (Fig. 1.3) cover user manuals (system overview, beginner's guide tutorial, reference guide) and operational manuals (installation guide, system administration guide). Their function: **"instructions to setup and use the software system and instructions on how to react to system failure."**

Difference from a program: "any program is a subset of software and it becomes software only if documentation and operating procedure manuals are prepared." Adding code never converts a program into software; only documentation and procedures do.

Aggarwal, §1.2.2, pp.4–5; Fig. 1.1 p.4, Fig. 1.2 p.5, Fig. 1.3 p.6.

1.15 · Discuss major areas of the applications of the software.

Answer

Eight areas (Fig. 1.7): **system software** — **"a collection of programs to provide service to other programs"** · **real time software** — monitors and analyses real-world events as they occur · **embedded software** — placed in ROM, controls the functions of a product, **"also termed as intelligent software"** · **business software** — **"the largest application area"** · **personal computer software** · **artificial intelligence software** — **"non-numerical algorithms to solve complex problems"** · **web based software** · **engineering and scientific software**.

Aggarwal, §1.3, pp.10–11; Fig. 1.7, p.10.

1.16 · Is software a product or process? Justify your answer with examples.

Answer

Product: "What is delivered to the customer, is called a product. It may include source code, specification document, manuals, documentation etc. Basically, it is nothing but a set of deliverables only." Process: "the way in which we produce software. It is the collection of activities that leads to (a part of) a product."

Applying the two definitions: the software artefact is the **product** — it is what is delivered — while designing, coding and testing are the **process** that produced it.

Aggarwal, §1.5.2, p.13.

1.17 · Differentiate between: (i) Deliverables and milestones (ii) Product and process (iii) Measures, metrics and measurement

(i) Deliverables and milestones. Deliverables are produced — source code, user manuals, operating procedure manuals. Milestones are **"the events that are used to ascertain the status of the project"**, e.g. finalisation of specification, completion of design documentation.

(ii) Product and process. Product = what is delivered to the customer. Process = the way in which we produce software.

(iii) Measures, metrics and measurement. A **measure "provides a quantitative indication of the extent, dimension, size, capacity, efficiency, productivity or reliability of some attributes of a product or process." Measurement** is **"the act of evaluating a measure."** A **metric** is **"a quantitative measure of the degree to which a system, component, or process possesses a given attribute."**

Aggarwal, §1.5.1–1.5.3, p.13.

1.18 · What is software metric? How is it different from software measurement?

Answer

Metric: "a quantitative measure of the degree to which a system, component, or process possesses a given attribute." Measurement: "the act of evaluating a measure." One is a quantity the system possesses; the other is the act of evaluating it.

Pressman [PRES02]: **"When a single data point has been collected (e.g., the number of errors uncovered in the review of a single module), a measure has been established. Measurement occurs as the result of the collection of one or more data points... A software metric relates the individual measures in some way (e.g., the average number of errors found per review)."**

Aggarwal, §1.5.3, p.13.

1.19 · Discuss software process and product metrics with the help of examples.

Process metrics "quantify the attributes of software development process and environment" — productivity, quality, failure rate, efficiency. **Product metrics "are measures for the software product"** — size, reliability, complexity, functionality.

Aggarwal, §1.5.4, p.13.

1.22 · Distinguish between generic and customised software products. Which category accounts for more development projects, and what proportion does the source report?

Generic products "are developed for anonymous customers. The target is generally the entire world and many copies are expected to be sold. Infrastructure software like operating systems, compilers, analysers, word processors, CASE tools etc. are covered in this category." Customised products "are developed for particular customers. The specific product is designed and developed as per customer requirements."

Which accounts for more development projects: customised products — **"Most of the development projects (say about 80%) come under this category."**

Aggarwal, §1.5.7, p.14.

⚠ **دقّق بالوحدة قبل ما تكتب**

نسبة 80% تخص مشاريع التطوير، ولا تحدد حصة السوق أو عدد المستخدمين والنسخ.

1.23 · Describe the role of management in software development with the help of examples.

"The management of software development is heavily dependent on four factors: People, Product, Process, and Project."

"Software development is a people centric activity. Hence, success of the project is on the shoulders of the people who are involved in the development."

Manager selection is **"most crucial and critical"**, because **"a good manager cannot ensure the success of the project, but can increase the probability of success." "Proper selection, training, compensation, career development, work culture etc." "I am more afraid of an army of one hundred sheep led by a lion than an army of one hundred lions led by a sheep."**

"All four factors (People, Product, Process and Project) are important for the success of the project. Their relative importance helps us to organise development activities in more scientific and professional way."

Aggarwal, §1.6 and §1.6.1, p.15; §1.6.4, p.16.

1.24 · What are various factors of management dependency in software development? Discuss each factor in detail.

Answer

"People, Product, Process, and Project." — dependency order (Fig. 1.8): **People (1) → Product (2) → Process (3) → Project (4)**.

People. "Software development is a people centric activity. Hence, success of the project is on the shoulders of the people who are involved in the development." Managers need **"mental toughness to endure inner pain"** and must **"plan for the best, be prepared for the worst, expect surprises, but continue to move forward anyway."** Manager selection is **"most crucial and critical"**; once a good manager is in place, **"project is in safe hands."**

Product. "A solution to his/her problems." A **"best"** approach is selected **"within constraints imposed by delivery deadlines, budgetary restrictions, personnel availability, technical interfaces etc."** Without well-defined requirements **"it may be impossible to define reasonable estimates of the cost, development time and schedule."**

Process. "The process is the way in which we produce software. It provides the framework from which a comprehensive plan for software development can be established." "If the process is weak, the end product will undoubtedly suffer." "The process priority is after people and product, however, it plays very critical role for the success of the project."

Project. "A proper planning is required to monitor the status of development and to control the complexity." "Most of the projects are coming late with cost overruns of more than 100%." "Warning before the occurrence of any surprise."

Aggarwal, §1.6 and §1.6.1–1.6.4, pp.15–16; Fig. 1.8, p.15.

1.25 · What is more important: Product or process? Justify your answer.

Answer

"An efficient process is required to produce good quality products. If the process is weak, the end product will undoubtedly suffer, but an obsessive over-reliance on process is also dangerous."

The focus should remain on product quality and the process used to develop it, with Product ranked before Process in the management dependency order.

Neither is universally more important: an effective process supports product quality, while excessive reliance on process can be harmful. The right balance depends on the product and its requirements.

Aggarwal, §1.5.2, p.13; §1.2, p.4; §1.6.3, p.16; Fig. 1.8, p.15.

ASE CHAPTER 1

T3 · Understand (Why / How)

Explain-level questions grounded in Aggarwal — answers in English, traps and notes in Arabic

Chapter 1 — Introduction

طريقة قراءة الإجابات

الكلام العريض مبني على المصدر، وقد يُبسّط بألفاظ أسهل. الكلام العادي صياغة أو استنتاج مختصر. الاقتباس الحرفي فقط يوضع بين علامتي تنصيص.

Rapid self-test — recall prompts (no answers shown)

Why did the goal shift from "working programs" to "maintainable programs"?
Give two software-failure examples from the chapter, and explain whether they establish a direct link to hardware progress.
State any three of the nine software myths and their effect on the process.
Give the four reasons software process improvement is difficult.
Define metric, measurement, and measure in one line each.

Retrieval set — T3 Understand (why / how)

U.1 · Why has the main goal of software development moved from simply producing good software to producing software that is also maintainable?

Answer

Because the cost of maintaining software increased, the objective is now to produce software that is maintainable, delivered on time, within budget, and that satisfies its requirements.

Aggarwal, §1.2.1, p.4.

U.2 · Why is the software crisis said to be made worse by the progress of hardware technology? Explain with examples.

Answer

Rapid hardware gains contrasted with slower software progress because software faces difficulties such as complexity, changeability, and invisibility. Y2K and Ariane-5 illustrate software failures; they do not by themselves prove that hardware progress caused the crisis.

Two examples from the failure list: **Y2K involved shortening the four-digit year to a two-digit format, such as 64; Ariane-5 suffered an overflow when the guidance computer converted sideways velocity from a 64-bit format to a 16-bit format.**

Aggarwal, §1.1.2, p.3; §1.4, p.12; §1.1.1, pp.2–3.

⚠ فخ هذا السؤال

لا تخلط بين تزامن تقدّم العتاد ومشكلات البرمجيات وبين إثبات علاقة سببية مباشرة. تغيّر نسبة كلفة العتاد إلى البرمجيات عامل مستقل يورده الكتاب ضمن أسباب الأزمة.

عبارة «practically non-existent» **تنطبق على Y2K فقط** — الكتاب يسمّي مشكلة Y2K نفسها كذلك، والضرر الحقيقي كان كلفة معالجة المشكلة. أما **Ariane 501 فكان فشلاً فعلياً** — خسِر فيه الصاروخ بسبب overflow التحويل من 64-bit إلى 16-bit — فلا تُعمّم وصف Y2K عليه.

U.3 · How do software myths interfere with the way software should be developed? Give the myths and their effects.

Answer

"There are number of myths associated with software development community. Some of them really affect the way, in which software development should take place." Nine myths and how each warps the process:

1. Software is easy to change. "Every change requires that the complete system be re-verified."

2. Computers provide greater reliability than the devices they replace. "Now, we have software error as well."

3. Testing or 'proving' software correct can remove all the errors. "Testing can only show the presence of errors. It cannot show the absence of errors."

4. Reusing software increases safety. "A false sense of security."

5. Software can work right the first time. "No aeronautical engineer would build a jet fighter without a prototype."

6. Software can be designed thoroughly enough to avoid most integration problems. "There is an old saying among software designers: 'Too bad, there is no compiler for specifications'."

7. Software with more features is better software. "The best, most enduring programs are those which do one thing well."

8. Addition of more software engineers will make up the delay. It **"may further delay the project."**

9. Aim is to develop working programs. "The aim has been shifted from developing working programs to good quality, maintainable programs."

Aggarwal, §1.4, pp.11–12.

U.4 · What is the software process, and why is it so hard to improve?

"The software process is the way in which we produce software."

Four reasons it is difficult to improve [HUMP89, WIEG99]: **not enough time** — unrealistic schedules leave no room to study what is wrong · **lack of knowledge** — developers do not read the literature on best-known practice · **wrong motivations** — initiatives launched to satisfy a contractor or a manager's CMM ambition · **insufficient commitment** — assessment without follow-through, no plan and no roadmap.

Aggarwal, §1.2.3, pp.6–7; Fig. 1.4, p.7.

U.5 · What is the difference between a software metric and a software measurement? Why does the distinction matter?

Answer

Metric: "a quantitative measure of the degree to which a system, component, or process possesses a given attribute." Measurement: "the act of evaluating a measure." One is the quantity a system possesses; the other is the act of evaluating it.

Pressman [PRES02]: **"When a single data point has been collected (e.g., the number of errors uncovered in the review of a single module), a measure has been established. Measurement occurs as the result of the collection of one or more data points... A software metric relates the individual measures in some way (e.g., the average number of errors found per review)."**

Aggarwal, §1.5.3, p.13.

تدقيق المطالبات (claim check) — هذا الملف

الكلام العريض يلخّص المادة المسندة للمصدر، والنص العادي يوضح الصياغة أو الاستنتاج. المراجع مثبتة تحت كل جواب.

ASE CHAPTER 1

T4 · Analyze (Documented Failures)

Failure-case analysis grounded in Aggarwal — answers in English, traps and notes in Arabic

Rapid self-test — recall prompts (no answers shown)

1. Name the six documented failures in the book's list and give the cause of any three.
2. What handling cost is reported for Y2K?
3. State the Patriot mechanism in one line (and the two uptime figures).
4. Why did Ariane-5's redundant unit fail to save the rocket?
5. Why did CASE tools fail to end the software crisis?

Retrieval set — T4 Analyze (documented failures)

A.1 · The chapter lists documented software failures. Name them, and state in one line what went wrong in each.

Y2K — the four-digit year was shortened to a two-digit format. · Patriot missile (Dhahran) — a small timing error in the system's clock accumulated. · a 1996 consumer database project was delivered on time but failed to work as promised; the design team was over-optimistic in agreeing to requirements, while developers became fixated on rebuilding and errors were ignored. · Ariane-5 — a 64-bit value converted to a 16-bit format caused an overflow. · financial software — faults produced wrong information or crashed the system. · Windows XP — 18 megabytes of patches were posted on the release day itself.

Aggarwal, §1.1.1, pp.2–3.

A.2 · Why did Y2K become a problem, how was the problem described, and what did handling it cost?

Y2K arose because only the **last two digits of the year** were used — the four-digit date was shortened to a two-digit form. The problem was described as **"practically non-existent"**, and handling it cost **"millions of rupees"**.

Aggarwal, §1.1.1, p.2.

A.3 · Analyse the Patriot missile failure: what was the root-cause mechanism, and why did long uptime matter?

The Patriot missiles **failed several times to hit Scud missiles**, including one that **killed 28 US soldiers in Dhahran, Saudi Arabia**. Root cause: **a small timing error in the system's clock accumulated until, after 14 hours, the tracking system was no longer accurate.** In the Dhahran attack the battery **had been operating for more than 100 hours**, so the small error had time to grow.

Aggarwal, §1.1.1, p.2.

⚠ لا تضف أرقام البتات

الكتاب يذكر **خطأ توقيت صغير يتراكم** و14 ساعة و100 ساعة فقط. تفاصيل 24-bit و0.1s و0.3433s مو موجودة عند أغروال؛ لا تنسبها إليه. ربط حجم الخطأ بتمثيله بالبتات يحتاج مصدراً إضافياً.

A.4 · Analyse the Ariane-5 failure: what went wrong, and why did the backup unit not save the mission?

📝 **Answer**

Ariane-5 **was destroyed 39 seconds after launch, with its four-satellite payload.** Root cause: the guidance computer converted the rocket's **sideways velocity from a 64-bit to a 16-bit format**; **the number was too big and an overflow occurred after 36.7 seconds.** The identical backup unit **had failed the same way a few milliseconds earlier**, so redundancy gave no protection — the designers had assumed **the value would never be large enough to cause trouble.**

Aggarwal, §1.1.1, pp.2–3.

A.5 · Analyse the Space Shuttle fuel-dump crash. Why did it happen, and what did the follow-up find?

📝 **Answer**

During an abort the Shuttle dumps fuel to lighten the craft. The fuel-management module **finished the first dump and exited**, then **restarted for the second dump — where the crash occurred.** A **simple fix solved it**, but applying a general method to that module and others **surfaced seventeen additional, previously unknown problems.**

Aggarwal companion slides, slides 14–15.

⚠ **سند الحالة = شرائح، مو صفحة كتاب**

هذي الحالة **مو بالسرد النثري للكتاب**. قائمة أغروال (§1.1.1، ص.2) هي: Patriot · Y2K · قاعدة بيانات 1996 · Ariane-5 · برمجيات مالية · Windows XP. الرحلة الشاتل جاية من **شرائح المؤلف الرسمية** (سلايد 14–15) — فاذكرها **كسلايد**، ولا تنسبها لصفحة كتاب.

A.6 · Why is there "no silver bullet"? Analyse why CASE tools did not end the crisis.

📝 **Answer**

There is **no single development — in technology or in management — that by itself promises even one order-of-magnitude improvement in productivity, reliability or simplicity.** CASE tools were tried and **companies spent large sums with highly unsatisfactory results**; the reason is that **the hard part is specifying, designing and testing the conceptual construct, not the labour of representing it.**

Aggarwal, §1.1.2, p.3.

A.7 · What common pattern do these failures share, and why do they matter to the discipline?

📝 **Answer**

These cases illustrate the software crisis and its effects, including failures that caused **the death of many human beings** and made the crisis **a fixture of everyday life. Software myths, poor quality, increasing cost, and delays in delivery** were driving forces behind the emergence of software engineering.

القاسم المشترك بين هذه الحالات — مشكلات برمجية تؤدي إلى فشل أو كلفة أو تأخير — قراءة تحليلية للمجموعة. **Aggarwal, §1.1, pp.1–2; §1.4, p.12.**

ASE CHAPTER 1

T5 · Evaluate

Judgement questions grounded in Aggarwal — answers in English, traps and notes in Arabic

Rapid self-test — recall prompts (no answers shown)

Give your reasoned position on software engineering as art, craft, or science.
Which product type is the larger share of development, and what exactly is the 80% measuring?
Explain why a sound process helps product quality and why over-relying on process can be harmful.
What is the management dependency order of the four factors?

Retrieval set — T5 Evaluate

E.1 · Is software engineering an art, a craft, or a science? Judge and justify your position.

Software engineering is best described as an **engineering discipline**: systematic in its principles and still dependent on design judgement. The **art of programming alone is no longer enough to build large programs**, and software engineering is an important discipline of study, practice and research.

Aggarwal, §1.2.1, p.4; §1.6.4, p.16.

E.2 · Which type of software product accounts for more development projects — generic or customised? Evaluate and justify.

Generic products are **"developed for anonymous customers"** — the target is **"the entire world"** and many copies are sold (operating systems, compilers, word processors, CASE tools). **Customised products** are **"developed for particular customers"** to their specific requirements. Judging by development volume, customised is larger: **"most of the development projects (say about 80%) come under this category."**

Aggarwal, §1.5.7, p.14.

⚡ **دقّق بالوحدة قبل ما تحكم**

الـ80% نسبة **من المشاريع المطوَّرة**، مو من المستخدمين ولا من النسخ. المنتج generic يُباع «للعالم كله» بنسخ كثيرة فيوصل مستخدمين أكثر.

E.3 · What is more important: product or process? Justify your judgement.

"An efficient process is required to produce good quality products. If the process is weak, the end product will undoubtedly suffer, but an obsessive over-reliance on process is also dangerous." The aim is to focus **"on both the quality of the product and on the process used to develop the product"**, and the dependency order puts process **"after people and product."**

Neither is universally more important: an effective process supports product quality, while excessive reliance on process can be harmful. The right balance depends on the product and its requirements.

Aggarwal, §1.5.2, p.13; §1.6.3, p.16; Fig. 1.8, p.15.

ASE CHAPTER 1

T6 · Create (Diagram)

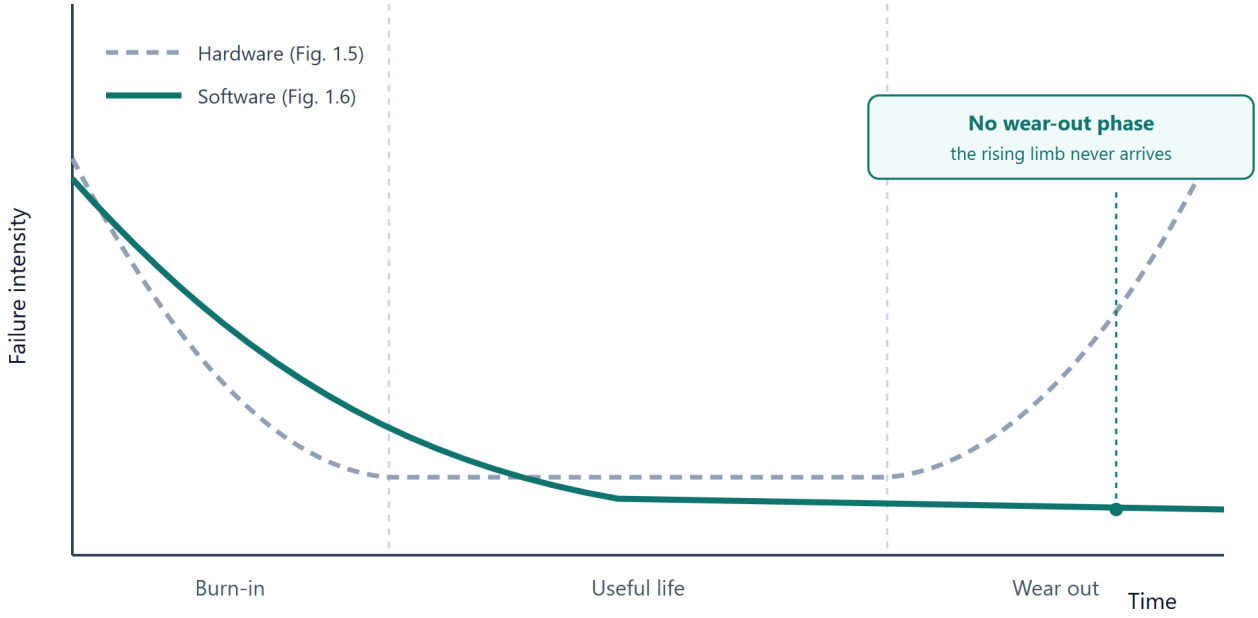
The software non-wear-out curve against the hardware bath-tub curve — answers in English, notes in Arabic

Retrieval set — T6 Create

D.1 · Draw the software failure-intensity curve against the hardware bath-tub curve, labelling the axes and the phases.

Answer

Axes: failure intensity (vertical) against time (horizontal). The hardware curve (Fig. 1.5) has three phases: high intensity during **burn-in**, a low flat **useful-life** plateau, then a **rising** limb at **wear out**. The software curve (Fig. 1.6) has **only the descending curve**: defects are found and fixed, so intensity keeps falling and the rising limb never arrives.



Failure intensity against time — hardware bath-tub (Fig. 1.5) vs software (Fig. 1.6)

Aggarwal, §1.2.4, pp.8–9; Fig. 1.5 p.8, Fig. 1.6 p.9.

D.2 · Interpret the drawing: why does the software curve never rise again?

Software does not wear out and has no wear-out phase. It **becomes obsolete when the environment it was developed for changes.**

Aggarwal, §1.2.4, pp.8–10.

Rapid self-test — recall prompts (no answers shown)

Name the three phases of the hardware curve.
What shape does the software curve take, and why?
What happens to software instead of wearing out?

ASE CHAPTER 1

T7 · Apply / Calculate

Productivity and effort — the chapter's only applied relationship — answers in English, notes in Arabic

Rapid self-test — recall prompts (no answers shown)

1. Define software productivity.
2. What is the appropriate unit of effort, and why person-months?
3. Compute LOC/PM for 48,000 LOC at 16 person-months.

Retrieval set — T7 Apply / Calculate

C.1 · Define productivity and state the appropriate unit of effort.

📝 **Answer**

Productivity is **"the rate of output, or production per unit of effort, i.e., the output achieved with regard to the time taken but irrespective of the cost incurred."** Output is measured in **lines of code (LOC)**. The appropriate unit of effort is **"Person Months (PMs), meaning thereby number of persons involved for specified months."**

Aggarwal, §1.5.5, p.14.

C.2 · Apply: a project delivers 30,000 LOC at an effort of 15 person-months. Compute the productivity.

📝 **Answer**

Productivity = output ÷ effort = **30,000 LOC ÷ 15 PM = 2,000 LOC/PM.**

الأرقام (30,000 و15) مثال تدريبي لتطبيق المعادلة؛ أما المعادلة والوحدة فمن المصدر. **Aggarwal, §1.5.5, p.14**

C.3 · Why is productivity expressed as LOC/PM rather than as a raw number of lines?

📝 **Answer**

Because it is a **rate** — output per unit of effort — its unit must combine output and effort, hence **LOC/PM**. A raw line count ignores how much effort produced it.

تفسير الوحدة بوصفها نسبة بين المخرَج والجهد هو توضيح مباشر للمعادلة؛ المصدر يحدد المخرَج والجهد ووحدة الجهد. **Aggarwal, §1.5.5, p.14**

ASE CHAPTER 1

V1 · Lecture-1 Vocabulary Coverage

The instructor's Lecture-1 terms mapped to their coverage — English terms, brief Arabic notes

Supporting-source answers for the two lecture gaps

11 · How are error, fault, system error, and failure related?

> 📝 **Answer**
>
> **A human error or mistake can introduce a system fault. When faulty code runs, it can create a system error, an incorrect system state. If the system then fails to provide the expected service, a system failure occurs. In software-process usage, defect and fault are synonyms, not two separate stages.**
>
> **Sommerville, §11.2, p.297, Fig. 11.3; Pressman, §8.4.1, p.203.**

12–13 · What does professional software ethics require, and which code covers it?

> 📝 **Answer**
>
> **Software engineers have responsibilities beyond technical work and should act ethically and responsibly. The joint Software Engineering Code of Ethics and Professional Practice has eight principles: Public, Client and Employer, Product, Judgment, Management, Profession, Colleagues, and Self.**
>
> **Sommerville, §1.2, pp.15–16.**

Rapid self-test — recall prompts (no answers shown)

1. Which two topics are missing from Aggarwal Chapter 1, and which supporting sources cover them? How is `defect` related to `fault`?
2. Where exactly are the IBM figures printed, and what are the three numbers?
3. Which failure cases are in the book's own list, and which one comes only from the slides?
4. What components does the source include in software?